Before Fractal Continual Learning:
An Implementation Audit of Recursive Structural Growth and Long-Horizon Evaluation

HUSHFUSION Forge Research

October 2026

Abstract

Continual learning is normally posed as parameter adaptation under a fixed computational graph; whether the graph itself should grow—structural plasticity—is a separate question, often asserted but rarely audited. We construct a recursive fractal-loop learner mixing root and descendant outputs through a normalized convex gate, with a relative-decay detector spawning descendants when improvement slows, and run it for 72,335 paired rounds in a non-stationary control environment against a fixed single-node learner. An implementation audit triggered by review reveals that in this run the descendant nodes were never trained: they appear in no optimizer's parameter table, and the advertised depth-dependent sparse schedule never enters a scheduling decision. The tree grew without its new components learning, so the apparent boundary result—"no structural advantage"—is not a test of a fractal learner but of a root node gating untrained random subnets. Long-horizon adaptation itself is confirmed for both root learners (warm cost falls from 130–176 to ∼9 steps; random baseline ∼32), and growth is operational (six events; one to seven nodes), but structural advantage is undecidable from this run. We report the audit, the repaired implementation, and the conditions for a decisive re-evaluation. Growth events are not evidence of advantage; reproducibility starts at the optimizer's parameter table.

1 Introduction

Continual learning is normally stated as a weight-update problem: keep the graph fixed and adapt parameters so that new tasks are learned without forgetting old ones [1, 3, 2]. This framing leaves an unasked question: when the task itself drifts, should the *organization of the updating loops* change, not just the weights inside them? The question matters for long-running controllers, where a learner must track a moving optimum for tens of thousands of rounds while preserving a stable response. Yet whether structural growth helps is nearly always asserted, not measured.

We treat it as a measurement problem, and the audit that follows is the paper's contribution. The architecture is a recursive fractal-loop learner: a dense root node, sparse descendants mixed through a normalized convex gate, and a relative-decay detector that spawns descendants when the improvement rate decays. The main experiment ran this learner for 72,335 paired rounds against a fixed single-node learner in a non-stationary control environment. A post-review code audit then found a decisive flaw in the main experiment's implementation: descendant nodes were never added to any optimizer's parameter table, and the advertised depth-scaled update schedule never entered a scheduling decision. The long run therefore evaluated a root node gating untrained random subnets, not a fractal learner. We report what remains valid (long-horizon adaptation is real; growth is operational), what is retracted (the claim of no structural advantage is undecidable from this run), and the repair plus validation plan that a decisive comparison requires. We release the audit trail and the repaired code.

Table 1: Long-horizon paired outcome (72,335 rounds). Warm rows are recommenders; cold rows are random controls. Exceedance = crossing the safety ceiling μ_{ceil}. Audit caveat: in this run the fractal descendants were never trained (§3.4), so the fractal column reflects a root gating untrained subnets, not a trained fractal learner.

Metric	Fixed	Fractal
Warm mean steps (full run)	11.1	11.5
Warm mean steps (converged, round $\geq$2000)	$\sim$9	$\sim$9
Cold mean steps	31.6	31.9
Exceedance rate	5.9%	5.8%
Final tree nodes	1	7

2 Results

2.1 Long-horizon continual adaptation succeeds in both root learners

Two learners, a fixed single-node network and a recursively grown fractal tree, were paired at every round on the same drifting initial condition μ_0 of a deterministic μ-dynamics controller (protocol in Methods). Each selected a control point from its accumulated experience and executed the environment. What the audit establishes is that, in this run, the "fractal" learner's descendants were not trained; the observable behavior is that of the root node gating untrained random subnets. With that caveat, the run still validates long-horizon adaptation for the root learners.

Both roots learned and kept learning. Warm performance fell from roughly 130–176 steps in the first 1,000 rounds to roughly 9 steps after 2,000 rounds, and stayed there for the remaining 70,000 rounds; the cold random baseline remained near 32 steps throughout (Table 1). Continual adaptation was therefore effective and stable, and it did not decay. This result is independent of the descendant-training flaw: it is a property of the root learner, and both roots exhibit it.

2.2 Structure grows, then stabilizes

The fractal tree grew six times (rounds 46, 57, 73, 80, 87, 94), from one to seven nodes, and then stopped. Growth was driven by a relative-decay detector (Methods): a descendant spawns when the exponentially weighted improvement rate of validation loss falls below a fraction of that structure's early improvement rate. After reaching seven nodes the detector never fired again for the remaining 72,000 rounds. Unbounded growth is not observed; the tree settles.

The audit complicates this picture. Growth is operational—the detector fires and the graph mutates—but the spawned descendants were not trained (Section 3.4). What the run demonstrates, therefore, is that the *detector and the graph mutation* work over a long horizon, not that the grown components learn. Growth events are real; their learning contribution was never exercised.

2.3 The structural advantage is undecidable: the implementation audit

The paper's earlier version claimed a boundary result: "no structural advantage." A post-review audit of the main experiment's implementation retracts that claim. Three findings are decisive.

Descendants are never trained. In `design_daemon_fractal.py`, each node's optimizer is constructed over the node's own network and gate parameters only ("opt = Adam(list(self.net.parameters()) + [self.gate])"). Children of a grown node are appended to `self.children`, but they never join any parent's optimizer parameter list, and the main loop calls

`train_step` only on the root. A descendant's parameters therefore never receive a gradient step. The only per-child operation is a warm-up counter decrement. The grown "tree" is a root network gating fixed, randomly initialized subnets.

The advertised sparse schedule never executes. The code computes `update_freq = D**depth` and stores it, but no scheduling decision anywhere consults it. Depth-dependent update frequency is documentation, not behavior.

The evaluation is not reproducible as stated. The 72,335-round trace and its analysis script are not in the repository; the daemon seeds its RNG from wall-clock time (`int(time.time()) % 10000`); a rerun does not reproduce the reported numbers. The repository's `README` acknowledges the trace as "live data, not committed."

Consequences. The paired long-run difference (11.5 vs. 11.1 warm steps) and the three-seed and 701-record comparisons all compare a trained root against a root gating untrained subnets—a condition that neither confirms nor refutes a fractal advantage. The correct statement is *undecidable from this run*. Long-horizon adaptation and growth detection remain valid; structural advantage requires a repaired implementation and a decisive re-run.

2.4 What the audit does not touch

Three earlier results are independent of the descendant-training flaw and remain valid. (i) Long-horizon continual adaptation is real: both roots converge from 130–176 to $\sim$9 warm steps against a $\sim$32 random baseline and hold for 72,000 rounds. (ii) The relative-decay detector is operational: six growth events, one to seven nodes, then stable. (iii) The control environment is a sound deterministic benchmark with a safety ceiling; this is unaffected by the learner implementation. These three stand; the structural-advantage claim falls.

3 Related Work

Our design sits at the meeting point of optimization-based meta-learning, which separates fast task adaptation from slower meta-parameter updates [1, 3]; continual-learning consolidation, which addresses forgetting under sequential tasks [2]; and fractal allocation arguments, which suggest that when marginal returns decline with depth, computation need not be spread evenly across levels—a principle with roots in Murray's minimum-work analysis of branching networks [6] and formalized in our companion Lean development [5]. We keep these as motives, not predictions. The present paper does not claim to resolve whether fractal organization helps; its contribution is to show how an apparently clean long-horizon comparison can silently fail to test the hypothesis, and what a decisive test requires.

4 Discussion

Two statements must be held apart, and the audit sharpens the separation. "Recursive structural plasticity is implementable and stable" is now qualified: growth events fire and the graph mutates, but in the audited run the grown components never learned, so we can only say the *detector and mutation* work, not the grown learner. "Structural growth improves performance" was the paper's earlier claim, and it is now retracted: the run did not test a trained fractal learner, so its null result is undecidable. Conflating "the tree grew" with "the grown components learned" is precisely the error this audit corrects; separating the two is the paper's methodological point.

The question of whether the advantage could appear at all remains open. The prefix-concentration argument that motivates fractal allocation [5] requires diminishing returns strong enough that concentrating budget in early levels beats spreading it. On a small learner with limited data, marginal returns may not decline sharply; splitting the budget across specialists may dilute each specialist's evidence. These are hypotheses, not findings: the audited run cannot speak to them because it never trained a fractal learner. The natural next test is to repair the implementation so that descendants actually train, then compare under matched compute and data, with pre-registered equivalence bounds and confidence intervals. Only then is the boundary decidable.

The practical lesson is procedural. Evaluate structural growth in three stages: (i) does the graph mutate safely, (ii) does the learner keep adapting after mutation, and (iii) does the mutation improve a pre-registered metric under matched compute and data? This paper ran stages (i) and (ii)—and the audit shows that stage (ii) was only partially satisfied (the graph mutates, but the grown components did not train). Stage (iii) was not reached, because the implementation flaw makes it undecidable. That stage is the correct next experiment.

5 Methods

5.1 Recursive fractal-loop architecture

The architecture is defined as follows; Section 3.4 audits what the deployed implementation actually did. Each node v holds a predictor $f_v(x;\theta_v)$ and may have descendants $\mathcal{C}(v)$. The node output is a convex combination,

$$y_v(x) = g_{v,0}\, f_v(x;\theta_v) + \sum_{u\in\mathcal{C}(v)} g_{v,u}\, y_u(x), \qquad g_{v,j} \geq 0, \quad \sum_j g_{v,j} = 1, \tag{1}$$

with gates produced by a softmax. Descendants are not trusted immediately: they are warmed independently before entering the parent's mixture, which prevents the "dead-child" failure where a zero-initialized gate gives a child zero contribution and therefore zero gradient.

The root is updated every round; a node at depth d updates at frequency D^{-d} for sparse factor D (we use $D = 4$). This concentrates computation in the shallow prefix—the operational analogue of a fractal allocation rule. As the audit shows, this sparse schedule was specified but not enforced by the deployed code.

5.2 Growth rule

Growth is triggered by relative decay of validation loss. Let $\bar{\imath}_t$ be an exponentially weighted improvement rate and $\imath_0$ the structure's early average improvement rate. A descendant is spawned when $\bar{\imath}_t/\imath_0$ stays below a threshold (0.25 in the deployed run) for several consecutive observations; the detector resets after growth. A clean-loss filter excludes safety-penalty values, because a penalty spike is not evidence of a structural bottleneck. We include the failure that motivated the design: an initial absolute rule ("loss no longer decreases") never fires, because a shallow model can improve forever at a slow rate.

5.3 Control environment

We assemble a deterministic μ-dynamics controller from the two-flow-ring model:

$$\mu_{t+1} = \mu_t + \eta_t(1-\mu_t), \qquad \eta_t = \eta_{\text{ext}} + (2\lambda - \lambda^2)(1-\mu_t), \tag{2}$$

where η_{ext} is an external drive and λ a convergence parameter for a candidate self-flattening term. The task is to reach $\mu \geq 0.999$ (work window) without crossing $\mu_{\text{ceil}} = 1 - m_e/m_i \approx 0.999780568$ (safety ceiling). Cost is the number of steps to the window; ceiling crossing is a failure. The environment is deterministic, has a clear safety boundary, and is non-stationary because a fresh $\mu_0 \sim U(0.01, 0.8)$ is drawn every round. It is a benchmark for method study, not evidence about any experimental antigravity device; λ remains an input gap in the upstream model.

5.4 Paired long-horizon protocol

Every round, both learners see the same μ_0. Each selects a control point from its experience (UCB over a 20×10 grid of $(\eta_{\text{ext}}, \lambda)$), executes the environment, and updates its parameters from a sliding window of recent experience (capacity 400). Odd rounds are cold random controls, even rounds are warm recommendations. The deployed run used 72,335 rounds, independent checkpoints every 10 rounds, and a supervisor that restarted the daemon on failure. The fractal tree was allowed to grow without a preselected depth, subject to a safety cap of seven nodes. As §3.4 details, the deployed implementation did not include descendant parameters in any optimizer, so the protocol description here is the intended design, not the executed behavior.

5.5 Failure analyses that shaped the design

Four failures are worth recording. (i) An absolute "no-improvement" growth detector never fires on shallow networks (they improve slowly forever); the relative-decay detector fixed this. (ii) Zero-initialized child gates produce dead children; uniform gate initialization plus a warm-up phase fixed this. (iii) Growing a layer without reinitializing per-parameter step-size meta-parameters damaged the fast-adaptation loop, so structural growth and meta-parameter growth cannot be treated as independent. (iv) A standard RSI indicator is inapplicable to a monotonically improving loss trajectory (one-sided deltas give RSI $\equiv$ 100), so the term is used here for "recursive self-improvement," not the financial indicator.

Data and code availability

The audited implementation, supervisors, and analysis scripts are in the HUSHFUSION Forge repository [4]; the audit findings in this paper cite the exact code locations (e.g. `design_daemon_fractal.py`, lines 132, 155, 341–351). The 72,335-round trace is live data on the server and is not committed; a repaired, seeded, reproducible re-run is the precondition for a decisive structural comparison. Writing standards are in `WRITING-STANDARD.md`.

References

[1] Chelsea Finn, Pieter Abbeel, and Sergey Levine. Model-agnostic meta-learning for fast adaptation of deep networks. In *Proceedings of the 34th International Conference on Machine Learning (ICML)*, pages 1126–1135, 2017.

[2] James Kirkpatrick, Razvan Pascanu, Neil Rabinowitz, Joel Veness, Guillaume Desjardins, Andrei A. Rusu, Kieran Milan, John Quan, Tiago Ramalho, Agnieszka Grabska-Barwinska, et al. Overcoming catastrophic forgetting in neural networks. *Proceedings of the National Academy of Sciences*, 114(13):3521–3526, 2017.

[3] Zhenguo Li, Fengwei Zhou, Fei Chen, and Hang Li. Meta-SGD: Learning to learn quickly for few-shot learning. *arXiv preprint arXiv:1707.09835*, 2017.

[4] Yuanjie Liu. HUSHFUSION Forge: design search, knowledge reuse, and continual-learning daemons. `https://github.com/logos-42/hushfusion-forge`, 2026. Long-horizon fracta-loop trace and reproduce scripts.

[5] Yuanjie Liu. LmPrinciple: Fractal allocation theorems for diminishing-return structures. `https://github.com/logos-42/lm-principle`, 2026. Lean formalization: Fractal.lean, prefix_allocation_optimal, fractal_beats_uniform.

[6] Cecil D. Murray. The physiological principle of minimum work: I. The vascular system and the cost of blood volume. *Proceedings of the National Academy of Sciences*, 12(3):207–214, 1926.